Aran Learn 6: Artificial Immune Systems

For: Inbathamizhan and Aasif. **Time:** about 6 hours with practice. **Before this:** Learn 1, and Learn 4 section 2 (false alarms and misses). No biology background needed.

Checksums and tokens prove what leaked. This layer answers a different question: is this data flow normal? It copies a trick from the human immune system so Aran can flag a leak pattern nobody has seen before, without a list of past attacks.

1. The biology in one page

Your body fights germs it has never met, and it has no list of every germ. It does this by learning what "self" looks like and treating anything else as suspect.

1. **T-cells are made with random detectors.** Each new T-cell carries a receptor (a sensor protein) of random shape, built by randomly shuffling gene segments. The body produces millions of different detectors without knowing any germ in advance.
2. **The thymus deletes the dangerous ones.** Young T-cells mature in the thymus, a small organ behind the breastbone. There they are shown the body's own proteins. Any T-cell whose receptor binds to "self" is destroyed. This is **negative selection**. Most young T-cells fail this test.
3. **Survivors only react to non-self.** Every self-reacting detector was removed, so the remaining T-cells bind only to things that are not part of the body, such as a virus.
4. **Memory.** After an infection, the T-cells that matched multiply and some stay for years, so a second attack by the same germ is stopped faster.
5. **When it goes wrong.** If negative selection lets a self-reactive cell through, the body attacks itself: an autoimmune disease. In Aran terms, that is a false alarm on normal traffic.

Biology	Aran
Self proteins	Normal data flows: which service sends which identifier type to which destination, how much and when
Random T-cell receptors	Random detector points in the space of flow features
Thymus deletes self-reacting cells	Detectors that match any normal flow are thrown away
T-cell binds a virus	A detector matches a new flow, so Aran raises an alert
Immune memory	Confirmed bad flows are kept as fast signatures
Autoimmune disease	False alarms on legitimate new flows

Why this suits a bank. Leaks are rare and each one looks different, so there are never enough labelled attack examples to train a normal classifier. But a bank has months of normal traffic. Negative selection needs only examples of normal.

2. Turning data flows into points

The algorithm works on points in a space of numbers, so the first job is to describe each data flow as a short list of numbers between 0 and 1. Aran's Record step (see the research doc) builds a flow graph: each edge says "service A sent N values of identifier type T to destination B during hour H".

2.1 Worked example: one flow becomes one point

Flow: the `statement-export` service sent 12,000 Aadhaar tokens to a new SFTP destination at 03:00.

Feature	Raw value	How it is scaled to 0..1	Scaled
Hour of day	3	hour / 23	0.13
Volume	12,000 records	log10(volume + 1) / 6, capped at 1	0.68
Identifier sensitivity	Aadhaar	lookup: mobile 0.3, PAN 0.6, card 0.8, Aadhaar 1.0	1.00
Destination age	first seen today	1 − min(days known, 365) / 365	1.00

Point: (0.13, 0.68, 1.00, 1.00).

Why the log scale. Volumes range from 1 to millions. On a plain scale every small flow would sit at 0.00 and look identical; log10 spreads them out so that 100 and 10,000 records are clearly different.

2.2 Choosing features

- Keep them **few**: under about 10. Section 5 explains why.
- Make each one **meaningful to an analyst**, so an alert can be explained ("new destination, 3 a.m., Aadhaar, 50 times the usual volume").
- **Scale** them all to 0..1 so no single feature dominates the distance.

3. The negative selection algorithm

Forrest, Perelson, Allen and Cherukuri turned the thymus into an algorithm in 1994 ("Self-nonself discrimination in a computer", IEEE Security and Privacy). The real-valued version below works directly on points like those in section 2.

3.1 The four steps

1. **Collect self.** Take normal flows from a quiet, trusted period as points S.
2. **Generate candidates.** Draw random points in the same space.
3. **Censor.** Keep a candidate only if it is farther than a radius r_self from every self point. Give each kept detector the largest radius that still stops r_self short of the nearest self point. Detectors with their own variable radius are called **V-detectors** (Ji and Dasgupta, 2004); they cover open space with far fewer detectors than fixed-size ones.
4. **Monitor.** For each new flow, if it falls inside any detector's radius, it is non-self: raise an alert.

3.2 Picture it in 2D

```
1.0 +-------------------------+
    |   (D)        (D)    (D) |   D = detectors, each a circle
    |        (D)              |       that stops short of self
    |  (D)         x <-- attack falls inside a detector: alert
    |      .....         (D)  |
    |     .......  (D)        |   . = self (normal flows)
    |      .....              |
    |  (D)        (D)    (D)  |
0.0 +-------------------------+
    0.0                    1.0
```

3.3 The code

```
        import numpy as np
        rng = np.random.default_rng(0)

        def train(self_pts, n_detectors=500, r_self=0.1):
            detectors = []
            while len(detectors) < n_detectors:
                c = rng.random(self_pts.shape[1])                    # step 2: random
candidate
                d = np.min(np.linalg.norm(self_pts - c, axis=1))     # distance to nearest
self point
                if d > r_self:                                       # step 3: does not
touch self
                    detectors.append((c, d - r_self))                # radius stops r_self
short of self
            return detectors

        def is_anomaly(x, detectors):                                # step 4
            return any(np.linalg.norm(x - c) < rad for c, rad in detectors)
```

`np.linalg.norm(self_pts - c, axis=1)` computes the straight-line (Euclidean) distance from c to every self point in one go; `np.min` picks the nearest.

3.4 Worked experiment

Self: 200 points scattered around (0.3, 0.3). Three test groups of 200 points each: more normal points from the same spread, "attack" points around (0.8, 0.8), and "near-edge" points around (0.45, 0.45), just outside normal. With 500 detectors and seed 0:

r_self	False alarms on normal	Attacks caught (far)	Near-edge caught
0.05	0 / 200	200 / 200	177 / 200
0.10	0 / 200	200 / 200	74 / 200
0.20	0 / 200	200 / 200	0 / 200

How to read this: obvious anomalies are caught at every setting. The self radius decides how close to normal an anomaly can be before it is missed. A small radius catches subtle drifts but, on noisier real data, also flags more slightly unusual normal flows. These numbers come from

one seed on toy data; your runs will differ a little, which is why you should always repeat with several seeds.

3.5 The two settings that decide the trade-off

Setting	Larger value means	Cost
r_self (self radius)	Fewer false alarms	More misses near the edge of normal
Number of detectors	Better coverage of non-self space	More memory and slower checks

Estimating coverage. Sample 10,000 random points, keep those farther than r_self from self (they are non-self), and count the share that fall inside some detector. That share estimates how much of non-self space you cover. Add detectors until it stops rising.

4. How Aran uses it

The immune layer watches the flow graph continuously. Each hour, every edge becomes a point and is checked against the detectors.

Situation	What the point looks like	Result
CRM sends 2,000 PAN tokens to the KYC service at 11:00, as every weekday	Inside the self cloud	No alert
A new SFTP destination receives 12,000 Aadhaar tokens at 03:00	Late hour, high volume, max sensitivity, brand-new destination	Alert, before a dump ever appears online
The analytics job suddenly sends 50 times its usual card-token volume	Volume coordinate far above normal	Alert
A new vendor is onboarded and starts receiving mobile numbers	New destination	Alert; analyst marks it as self, and it is added to the self set

Immune memory in Aran. When an analyst confirms an alert as a real leak, the flow's point is stored as a signature. Future flows close to it raise a high-priority alert at once, without waiting for the hourly check.

How it fits with the other layers. The immune layer says "this flow is unusual". The checksum-complement tokens and per-sink attribution say "this is exactly what leaked, and from where". Together: early warning plus proof.

5. Known limits, so you can defend the design

- **High dimensions.** With many features, the space grows so fast that random detectors cover it poorly, and almost every point is far from every other (the "curse of dimensionality"). Keep features few and meaningful.
- **Self changes.** New vendors and products are legitimately new. Retrain self on a rolling window (for example the last 30 days), and let analysts mark a flow as self.
- **Poisoned self.** If an attacker is already leaking during the training period, the leak becomes "normal". Train on a period that was reviewed, and keep the rule-based checks running alongside.

- **Not a replacement.** The immune layer finds unusual flows; it does not prove anything leaked. That is the job of the checksum and tokens.
- **Compare fairly.** Always report results next to a standard baseline on the same data. In the toy experiment above, scikit-learn's Isolation Forest with default settings caught every attack and near-edge point but also flagged 43 of 200 normal points; negative selection at r_self = 0.05 flagged none of the normal points and caught 177 near-edge. One toy run proves nothing in general, which is exactly why the comparison must be repeated on real data.

6. Real data for testing

The public BGL supercomputer log in Loghub has real labels: lines starting with "-" are normal and the others are real alerts (about 140 of the 2,000 sample lines). It is not banking data, but it is real and labelled, so it shows honestly whether the algorithm separates normal from abnormal.

Suggested features per BGL line: hour of day, message length, count of digits, and whether the component field is one of the common ones. Train self on normal lines from the first half of the file, test on the second half, and report precision and recall for the alert lines.

7. Practice

- **P1.** Run the code in 3.3 with the setup of 3.4 and reproduce the table. Then run with seeds 1 to 5 and report the average and range.
- **P2.** Implement the coverage estimate from section 3.5 and plot coverage against the number of detectors (100, 250, 500, 1,000).
- **P3.** Write a `flow_to_point` function for the four features of section 2.1 and test it on three flows you invent.
- **P4.** Turn the BGL sample into feature points as in section 6, train on normal lines, and report precision and recall.
- **P5.** Compare your detector with `sklearn.ensemble.IsolationForest` on the same BGL points, and try setting its `contamination` parameter to the true alert share.
- **P6.** Add immune memory: keep confirmed attack points and flag any new point within 0.05 of one. Test it on a repeat of the attack group.

8. Further reading

- Forrest, Perelson, Allen and Cherukuri, "Self-nonself discrimination in a computer" (IEEE S&P 1994).
- Ji and Dasgupta, "Real-valued negative selection algorithm with variable-sized detectors" (GECCO 2004).
- Dasgupta and Niño, "Immunological Computation" (CRC Press, 2008), chapters 1 to 3.
- Liu, Ting and Zhou, "Isolation Forest" (ICDM 2008), for the baseline.
- Khan Academy, "Adaptive immunity", for the biology of T-cells and the thymus.